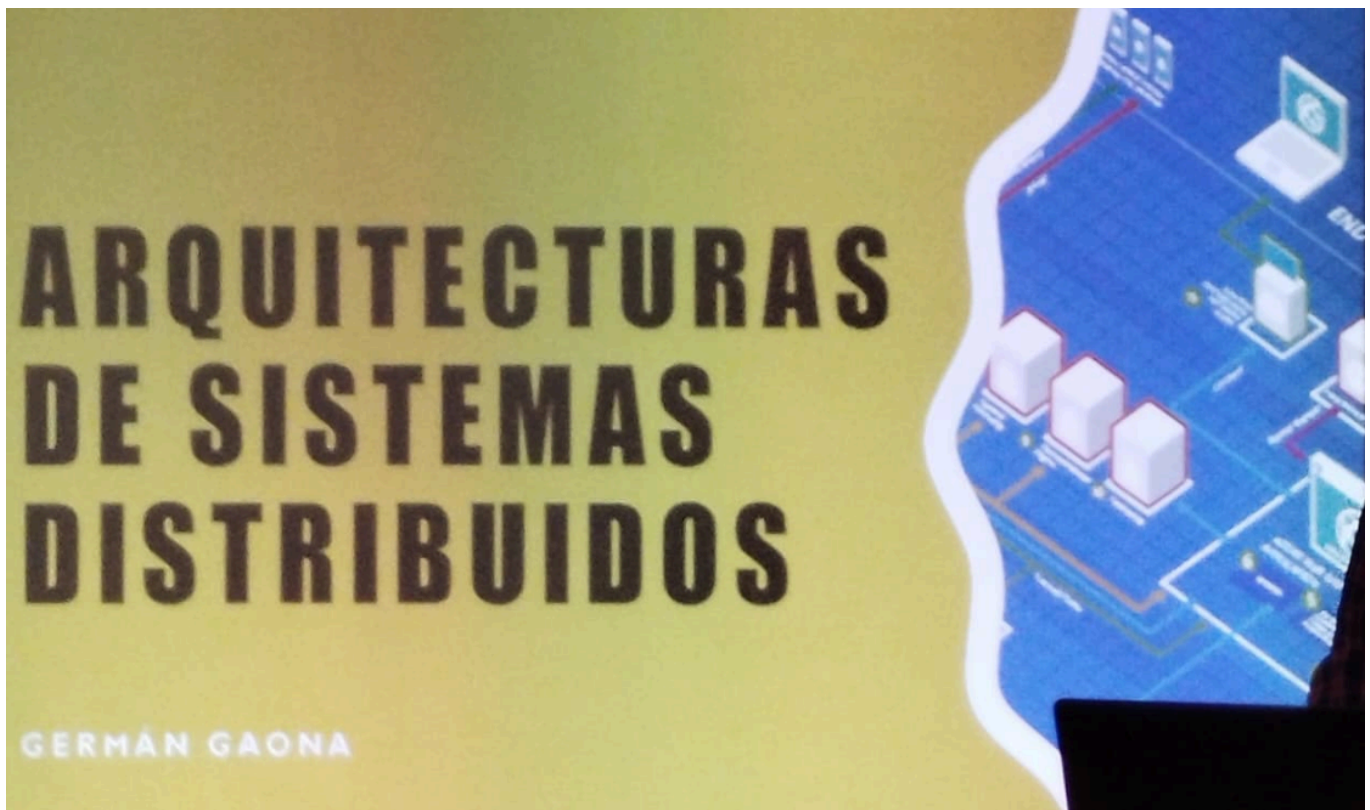


ARQUITECTURA DE SISTEMAS DISTRIBUIDOS



SISTEMAS DISTRIBUIDOS

SISTEMAS DISTRIBUIDOS

- "Una colección de ordenadores independientes que aparece ante el usuario como un único sistema coherente".
- Algunas ventajas:
 - **Uso compartido** de recursos: HW/SW
 - **Apertura**: sistemas diseñados en torno a estándares. Combinación de proveedores (HW/SW).
 - **Concurrencia**: los procesos pueden funcionar simultáneamente en equipos separados. Los procesos pueden comunicarse entre sí.
 - **Escalabilidad**: aumentar las capacidades del sistema añadiendo nuevos recursos. En la práctica, la vinculación de red puede limitar la escalabilidad.
 - **Tolerancia a fallos**: disponibilidad de varios equipos y posibilidad de replicar información.

- empezamos hablar de diferentes nodos funcionando en coordinación, debe ser transparente para el usuario
- suelen ser sistemas más complejos

Ventajas

- necesito compartir recursos, que pueden ser valiosos y necesito distribuirlos
- puedo tener distintas computadoras a la misma red. Pueden venir de distintos fabricantes (si fueran de la misma, sería muy fácil)
- cuando hablamos de sistemas distribuidos debemos tener una red de por medio, muchas veces están distribuidos geográficamente

Desventajas

SISTEMAS DISTRIBUIDOS

- Algunas desventajas:

- **Complejidad:** difícil de **entender** sus propiedades emergentes y de probar.
 - p. ej., el rendimiento dependiente de un procesador vs dependencia del ancho de banda de la red y de varios procesadores.
- **Seguridad:** acceso desde varios ordenadores diferentes, espionaje del tráfico de red. Es difícil garantizar la integridad de los datos. Degradación por ataques de denegación de servicio.
- **Manejabilidad:** diferentes tipos de computadoras, diferentes versiones del sistema operativo. Los fallos pueden propagarse a otras máquinas. Se requiere más esfuerzo para mantener el sistema en funcionamiento.
- **Imprevisibilidad:** La respuesta depende de la carga global del sistema, de su organización y de la carga de la red.

-al momento de probar algo o tratar de entender que esta sucediendo tengo que agregar muchos factores

-la computadora más segura es aquella que no está conectada a la red. En estos casos donde estoy compartiendo recursos todo el tiempo, en cualquier momento puede haber alguien que se quiera meter

-los recursos son más difíciles de gestionar, tengo que estar diferentes versiones de sw, de hw, etc

-nosotros no nos enteramos que existen los administradores IT hasta que hay un problema

Del libro:

-Son mucho más complejos que los sistemas centralizados y mucho más difíciles de diseñar, implementar y mantener

Algunos puntos a tener en cuenta para el diseño son:

1. Transparencia: ¿Hasta qué punto debe el sistema distribuido presentarse al usuario como un único sistema?

¿Cuándo es útil que los usuarios comprendan que el sistema está distribuido?

2. Apertura: ¿Debe diseñarse un sistema utilizando protocolos estándar que admitan la interoperabilidad, o deben utilizarse protocolos más especializados?

Aunque los protocolos de red estándar se utilizan universalmente, no ocurre lo mismo en niveles superiores de interacción, como la comunicación de servicios.

3. Escalabilidad: ¿Cómo se puede construir el sistema para que sea escalable? Es decir, ¿cómo se puede diseñar el sistema en su conjunto para que su capacidad pueda aumentarse en respuesta al incremento de la demanda?
4. Seguridad: ¿Cómo se pueden definir e implementar políticas de seguridad útiles que se apliquen a un conjunto de sistemas gestionados de forma independiente?
5. Calidad del servicio: ¿Cómo se debe especificar la calidad del servicio que se ofrece a los usuarios del sistema y cómo se debe implementar el sistema para ofrecer una calidad de servicio aceptable a todos los usuarios?
6. Gestión de fallos ¿Cómo se pueden detectar, contener (de forma que tengan efectos mínimos en otros componentes del sistema) y reparar los fallos del sistema?

Para diseñar sistemas distribuidos deberíamos crear abstracciones que oculten cómo el sistema está formado para evitar vulnerabilidades.

Se quiso adoptar el CORBA como estándar para sistemas distribuidos pero no tuvo éxito. Las compañías prefirieron sus propios estándares.

Muchos desarrolladores optaron por utilizar protocolos RESTful, pero este no está estandarizado.

La escalabilidad de un sistema refleja su capacidad para brindar un servicio de alta calidad a medida que aumentan las demandas sobre el sistema. Las tres dimensiones de la escalabilidad son el tamaño, la distribución y la capacidad de gestión.

1. Tamaño: Debería ser posible añadir más recursos a un sistema para hacer frente al creciente número de usuarios. Idealmente, a medida que aumenta el número de usuarios, el sistema debería aumentar de tamaño automáticamente para gestionar dicho aumento.
2. Distribución: Debería ser posible dispersar geográficamente los componentes de un sistema sin degradar su rendimiento. A medida que se añaden nuevos componentes, su ubicación no debería ser un factor determinante. Las grandes empresas suelen utilizar recursos informáticos en sus diferentes instalaciones alrededor del mundo.
3. Gestionabilidad: Debería ser posible gestionar un sistema a medida que aumenta de tamaño, incluso si partes del sistema se encuentran en organizaciones independientes. Este es uno de los mayores desafíos de la escalabilidad, ya que implica que los gerentes se comuniquen y acuerden políticas de gestión. En la práctica, la gestionabilidad de un sistema suele ser el factor que limita su grado de escalabilidad.

Algunos tipos de ataques que podrían sufrir los sistemas distribuidos.

1. Intercepción: un atacante intercepta las comunicaciones entre partes del sistema, provocando una pérdida de confidencialidad.
2. Interrupción: los servicios del sistema son atacados y no pueden prestarse como se espera. Los ataques de denegación de servicio (DoS) consisten en bombardear un nodo con solicitudes de servicio ilegítimas, impidiendo procesar las solicitudes válidas.
3. Modificación: un atacante obtiene acceso al sistema y modifica datos o servicios del sistema.
4. Fabricación: un atacante genera información que no debería existir y la utiliza para obtener privilegios. Por ejemplo, un atacante puede generar una contraseña falsa y usarla para acceder al sistema.

La principal dificultad en los sistemas distribuidos radica en establecer una política de seguridad que pueda aplicarse de forma fiable a todos los componentes del sistema. Como

expliqué en el Capítulo 13, una política de seguridad define el nivel de seguridad que debe alcanzar un sistema. Los mecanismos de seguridad, como el cifrado y la autenticación, se utilizan para hacer cumplir dicha política. Las dificultades en un sistema distribuido surgen porque distintas organizaciones pueden ser propietarias de partes del sistema. Estas organizaciones pueden tener políticas y mecanismos de seguridad incompatibles entre sí. En consecuencia, puede ser necesario hacer concesiones en materia de seguridad para permitir que los sistemas funcionen conjuntamente.

La calidad de servicio (QoS) que ofrece un sistema distribuido refleja su capacidad para prestar sus servicios de forma fiable, con un tiempo de respuesta y un rendimiento aceptables para sus usuarios. Idealmente, los requisitos de QoS deberían especificarse con antelación y el sistema debería diseñarse y configurarse para ofrecer dicha QoS. Desafortunadamente, esto **no siempre es factible** por dos razones:

1. Puede que no sea rentable diseñar y configurar el sistema para ofrecer una alta calidad de servicio durante los picos de carga. La alta demanda puede implicar la necesidad de muchos servidores adicionales para garantizar que se mantengan los tiempos de respuesta. Este problema se ha mitigado con la llegada de la computación en la nube, donde los servidores se pueden alquilar a un proveedor durante el tiempo que se necesiten. A medida que aumenta la demanda, se pueden añadir servidores automáticamente.
2. Los parámetros de calidad de servicio pueden ser mutuamente contradictorios. Por ejemplo, una mayor fiabilidad puede implicar una menor capacidad de procesamiento, ya que se introducen procedimientos de verificación para garantizar que todas las entradas del sistema sean válidas.

La calidad del servicio es especialmente importante cuando el sistema maneja datos críticos en tiempo real, como transmisiones de audio o video.

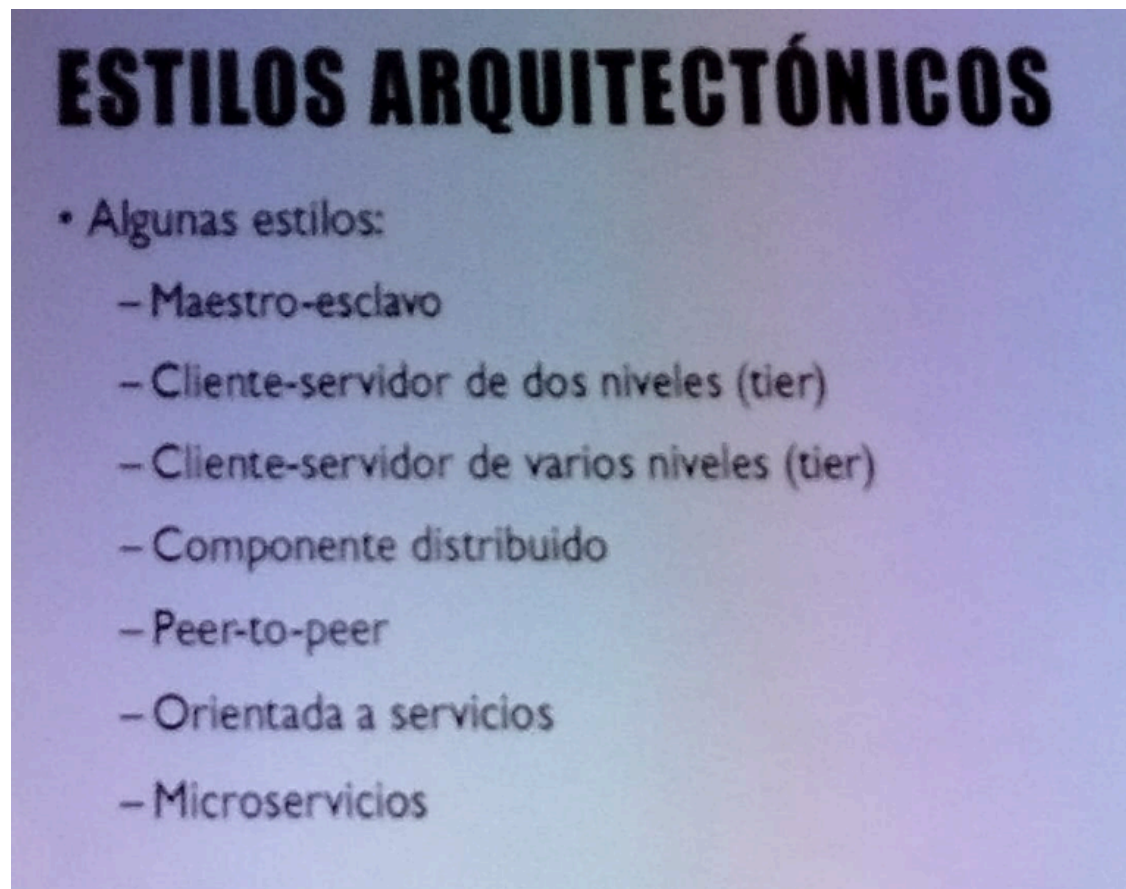
En un sistema distribuido, es inevitable que se produzcan fallos, por lo que el sistema debe diseñarse para ser resistente a estos fallos.

El **middleware** es software que actúa como una capa de comunicación entre aplicaciones, sistemas operativos y bases de datos

En un sistema distribuido, el middleware proporciona dos tipos distintos de soporte:

1. Soporte de interacción, donde el middleware coordina las interacciones entre los diferentes componentes del sistema. El middleware proporciona **transparencia de ubicación**, ya que no es necesario que los componentes conozcan la ubicación física de otros componentes. También puede admitir la conversión de parámetros si se utilizan diferentes lenguajes de programación para implementar componentes, la detección de eventos, la comunicación, etc.
2. Provisión de servicios comunes, donde el middleware proporciona implementaciones **reutilizables** de servicios que pueden ser requeridos por varios componentes del sistema distribuido. Al utilizar estos servicios comunes, los componentes pueden interoperar fácilmente y proporcionar servicios de usuario de manera consistente.

ESTILOS ARQUITECTÓNICOS



(pueden ser patrones tmb)

-Maestro-Esclavo: se utilizaba mucho en el tema de la base de datos

-componente distribuido y microservicios fueron haciendo una evolución

-lo comenta para que se sepa de donde vienen, no quiere decir que todos se utilizan hoy en día

MAESTRO-ESCLAVO

MAESTRO-ESCLAVO

- Proceso **maestro**: responsable de la computación, coordinación y comunicación.
- Procesos **esclavos**: dedicados a acciones específicas, como la adquisición de datos.
- Adecuado para el procesamiento distribuido predecible, requisitos de alto rendimiento, tolerancia a fallos o precisión.
- Común en sistemas de tiempo real con plazos de procesamiento. También en replicación de BD
- Los procesadores esclavos se encargan de las operaciones computacionalmente intensivas, como el procesamiento de señales y la gestión de equipos.

-hay un componente central que es el maestro

-las componentes o partes, acá habla de procesos. Estamos hablando como mínimo un proceso o servicio corriendo de manera separada

-esa separación puede estar separada en la misma memoria o separada directamente en distintas computadoras, lo esencial es que son 2 procesos distintos

-el libro lo plantea como un patrón para tratar de aumentar el rendimiento. En el cual los procesos esclavos se encargan de funcionamiento

-es mas que nada por tener los datos distribuidos en diferentes lugares, ya sea por confiabilidad o por cuestión de performance para tener datos duplicados en diferentes lugares

-los esclavos van enviando información al maestro y cada tanto el maestro a veces va enviando información a los esclavos

-la idea es que tenga tolerancias específicas

-los esclavos no necesariamente se conocen entre sí

EJEMPLO

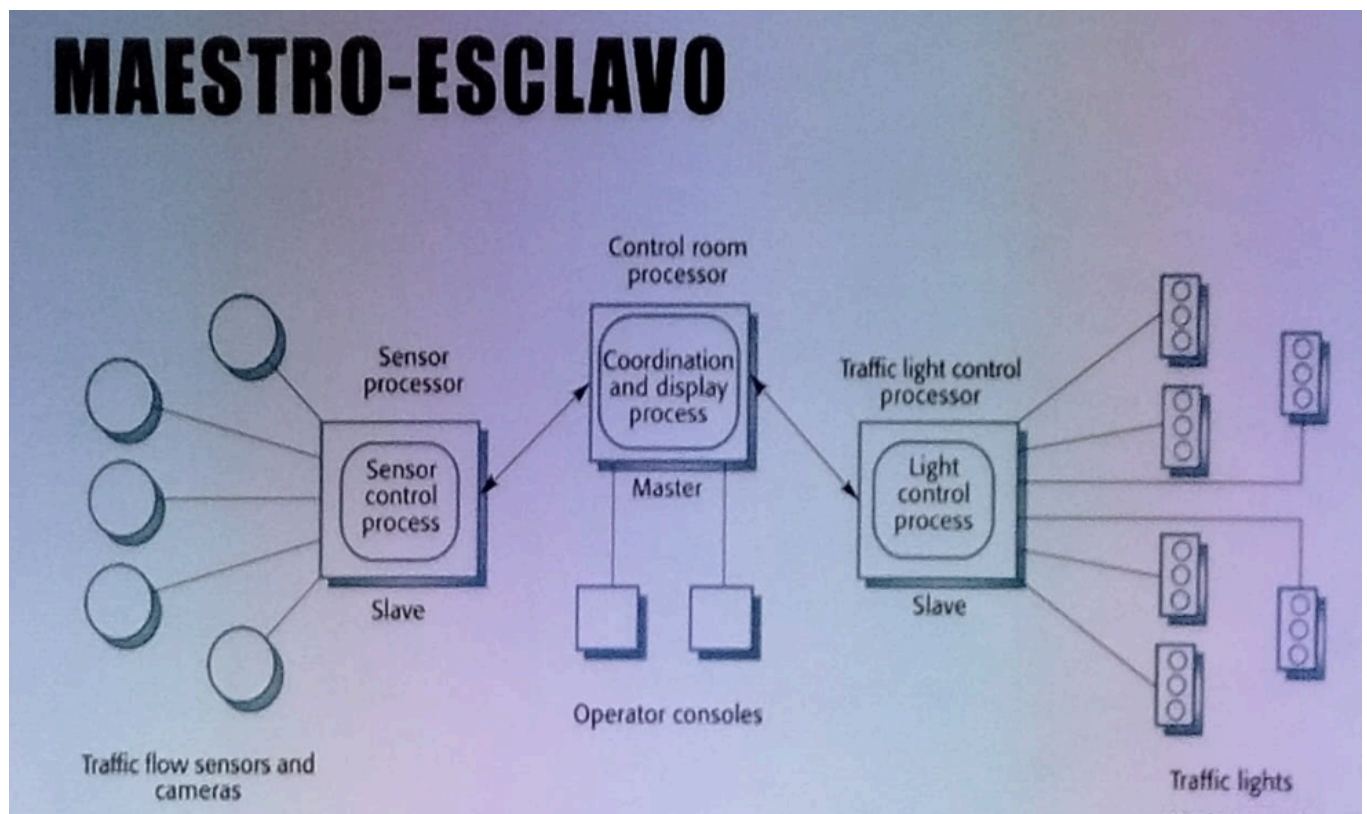
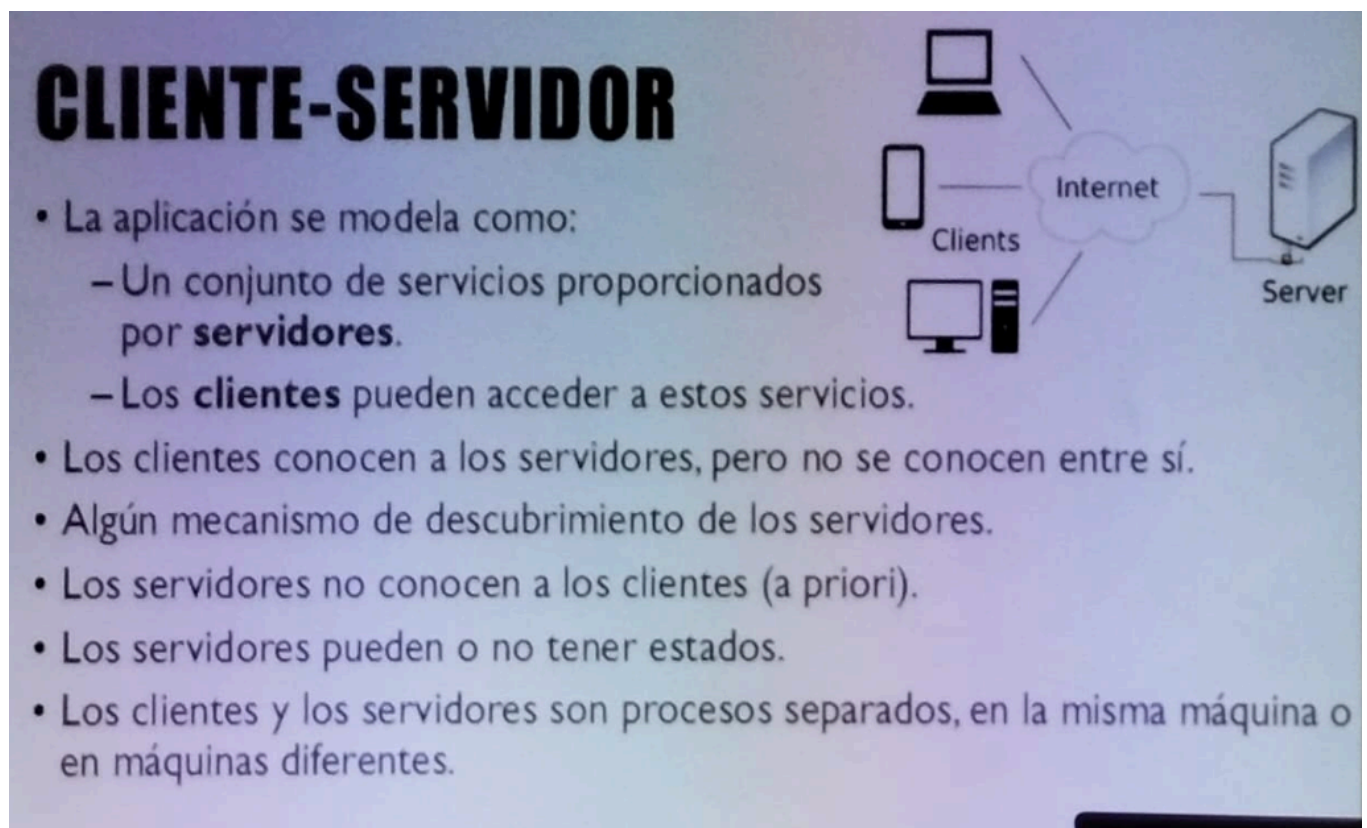


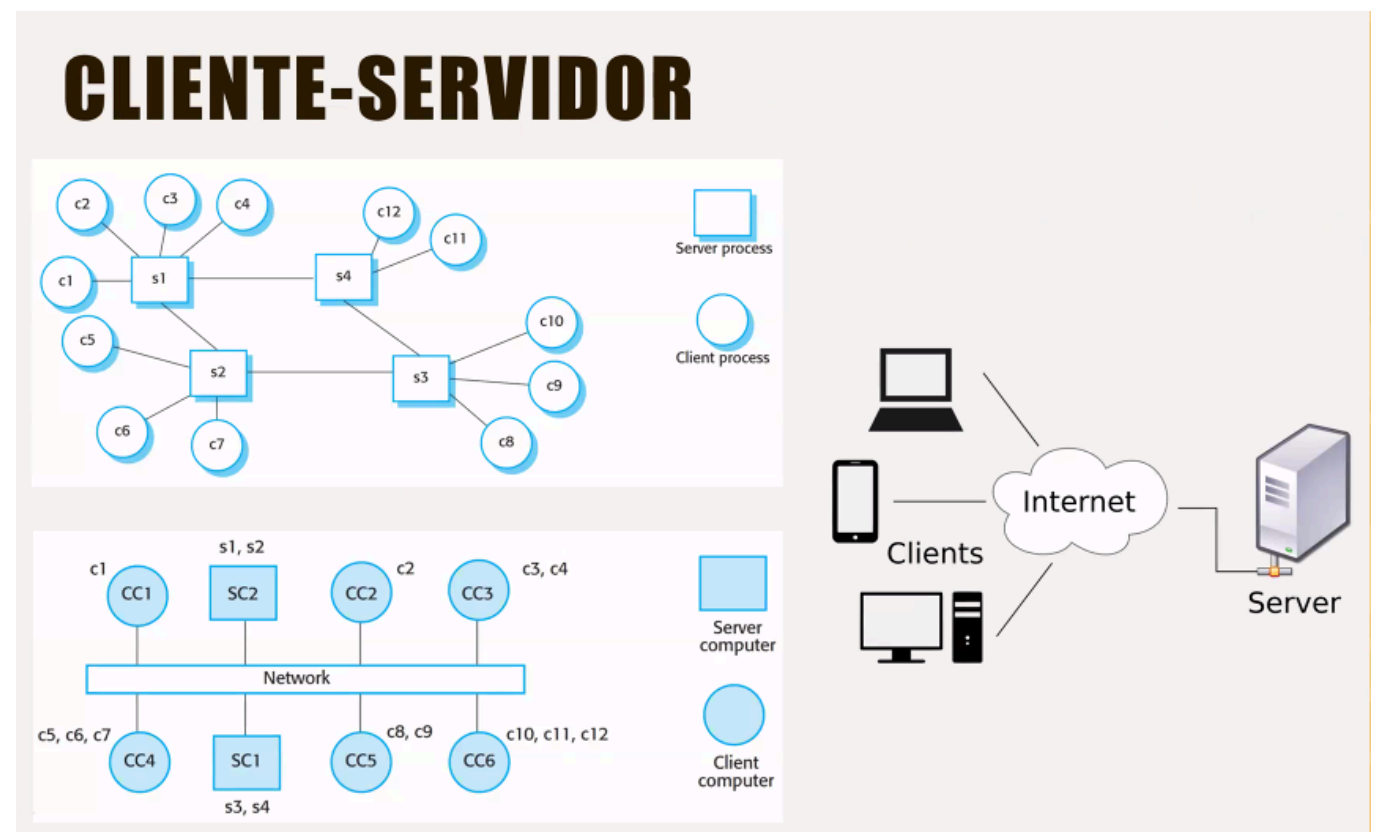
Figure 17.7 A traffic management system with a master–slave architecture

CLIENTE-SERVIDOR



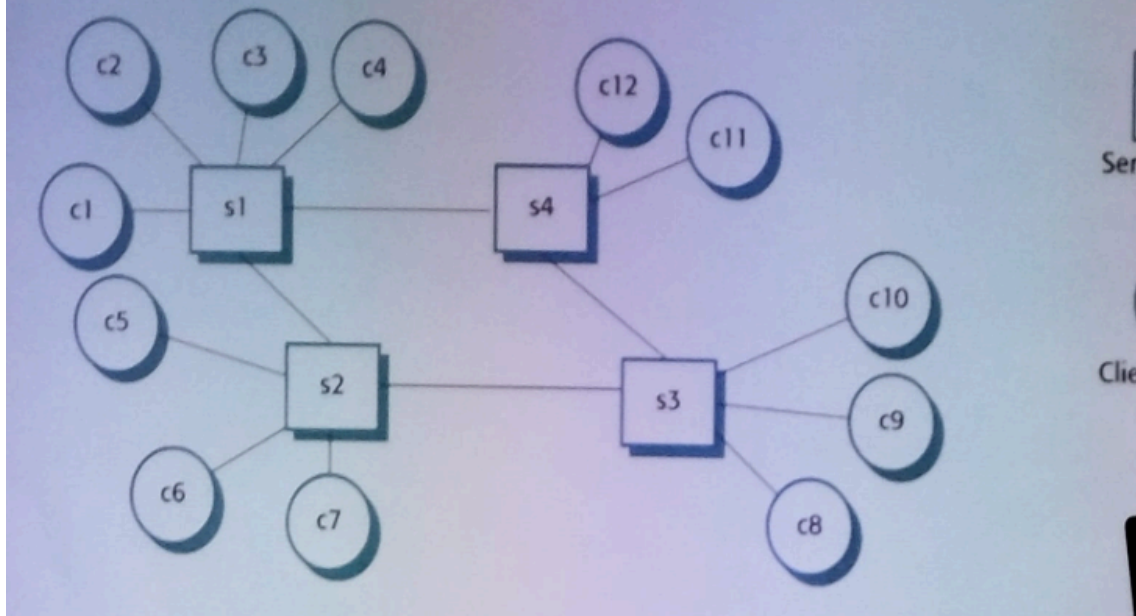
- los clientes son los que se conectan al servidor
 - aunque parezca un esquema similar al anterior, en este caso es al revés. Los demás consumen sus servicios y los controlan
 - los clientes no se conocen entre sí
 - los servidores pueden o no tener estados, originalmente cuando se hablaba de servidores de aplicaciones web se los pensó para que no tengan estados. Hace lo que tiene que hacer y luego se olvidó. A medida que fue avanzando se adquirió que recuerde lo que hace el cliente
 - separación física
 - Es eficiente porque aplica un algoritmo de balanceo de carga en momentos de muchas transacciones.
- cuando se habla de arquitecturas en capas se habla de una separación lógica, no necesariamente física

EJEMPLO

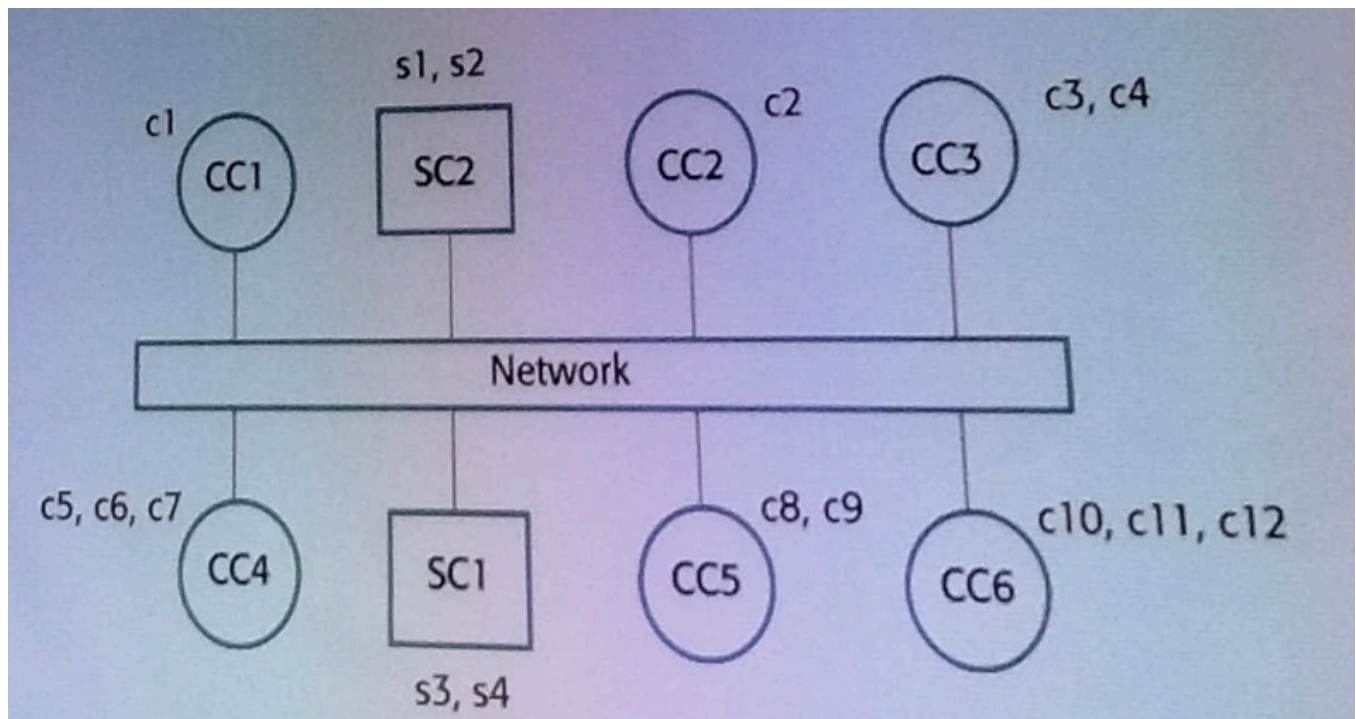


-son diagramas pero no son UML

CLIENTE-SERVIDOR



- son instancias de proceso que están interactuando
- muestra cada proceso a que servidor se comunica

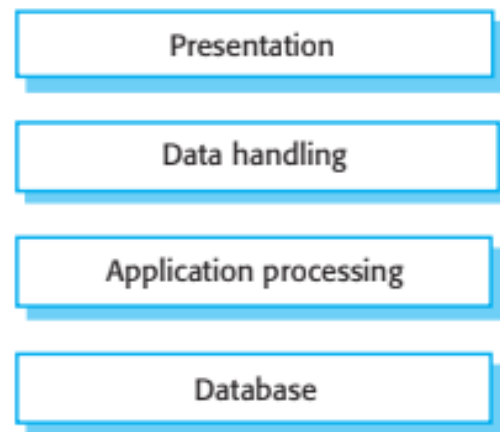


- este diagrama contempla nodos(ya sean físicos o virtuales)
- tenemos 4 servidores
- tenemos 12 clientes

- los servidores van tener cierta robustez para estar atendiendo todo el tiempo los servicios que se solicitan
 - este diagrama oculta a qué servidor está asociado cada proceso
 - están conectados a un red común pero no indica quien conoce a quien
- IMPORTANTE:** Nunca podemos modelar una arquitectura con un solo diagrama, ya que cada vista puede mostrar algo diferente para priorizar cierta información por sobre otra.

Este diagrama vendría a ser un diagrama de despliegue. Porque muestra computadoras (no solo clientes), servidores físicos (que alojan más de un servidor lógico) y como se relacionan a través de la red

Figure 17.6 Layered architectural model for client-server application



CLASE - 17/09 (clase virtual)

la arquitectura debe soportar la funcionalidad del sistema

(pregunto que es una petición HTTP a forma de repaso)

(el capítulo 17 de Somerville edición 10 tiene la mayoría de información)

(dps va a subir otro capitulo de otro libro basado en este, que no es el mismo)

C/S DE 2 NIVELES (2-TIER)

C/S DE 2 NIVELES (2-TIER)

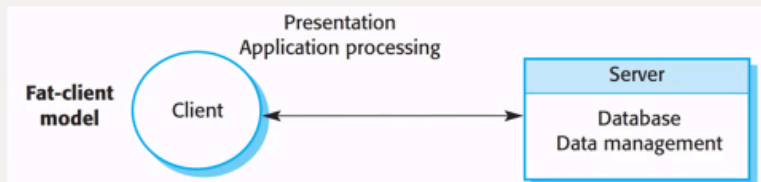
- Un servidor – varios clientes
- Dos formas:
- **Cliente liviano:** capa de presentación en cliente, el resto en el servidor.

- Fácil gestión de clientes
- Gran carga de procesamiento en el servidor y la red



- **Cliente pesado:** capa de presentación + aplicación en cliente, datos en servidor.

- Hace uso de la potencia de procesamiento disponible en el cliente.
- Se necesita desplegar y mantener el software en el cliente.



-la arquitectura cliente-servidor puede contener otras arquitecturas

-puedo tener varios niveles. Uno de los niveles más bajos es tener 2 capas

-en ambos casos la arquitectura la estoy manejando con 2 capas/niveles

-**cliente liviano** es aquel que no tiene mucho procesamiento, es muy básica la capa de procesamiento. Todo lo que sea web al menos empezó así, pero en esencia no sabemos lo que tenemos porque cada empresa se implementa de diferente forma

-la web se extendió

-**cliente pesado** cada vez que yo tengo que actualizar ese programa/aplicación tengo que actualizarlo de manera consciente y desplegarlo en varias máquinas. Los juegos son ejemplos de cliente-pesado moderno.

-al margen de la cantidad de capas lógicas que pueda tener, tengo 2 capas físicas como mínimo en el cliente-servidor

C/S MULTINIVEL (MULTI-TIER)

- Diferentes capas (layers), son separadas en diferentes procesos (tier) que pueden ejecutarse en diferentes procesadores.
- Usable en:
 - Aplicaciones a gran escala con cientos/miles de clientes.
 - Aplicaciones en las que tanto los datos como la aplicación son volátiles.
 - Aplicaciones en las que se integran datos de múltiples fuentes.
- Escalable:
 - Cada nivel puede gestionarse independientemente.
 - Pueden agregarse servidores adicionales conforme aumenta la carga

-la clave de los sistemas distribuidos aparte de compartir datos, sino también compartir recursos

-la idea era tener muchos clientes que se conecten a una **misma** base de datos

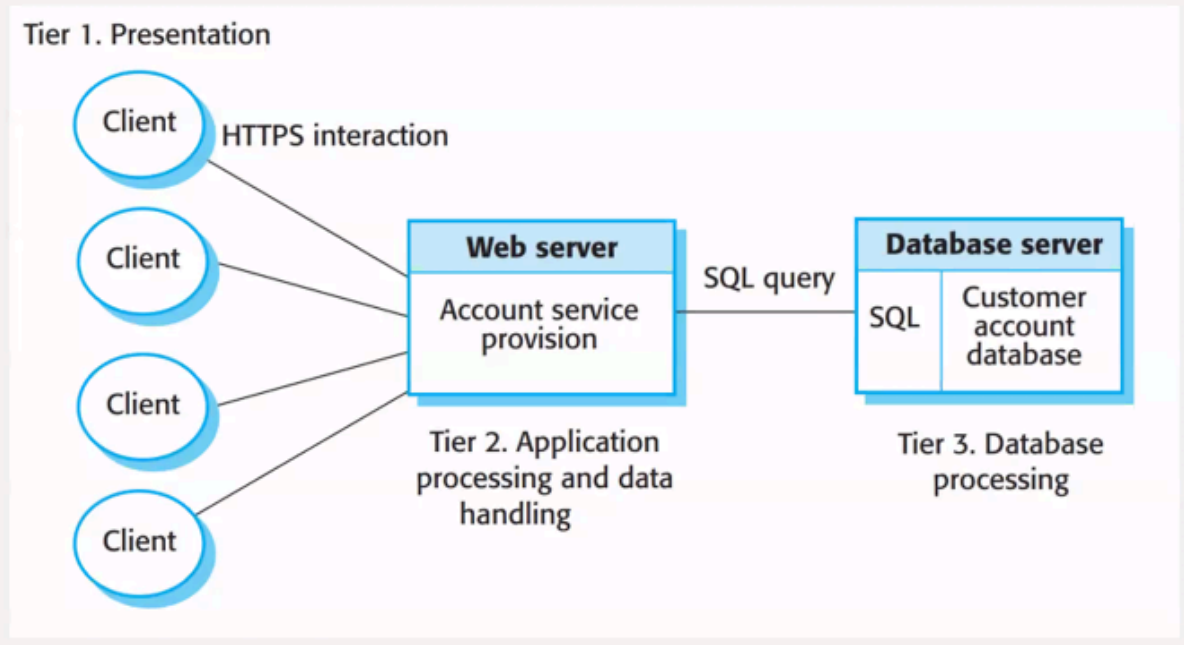
-esto lo que permitió es seguir dividiendo esta capa de cliente-servidor en varios niveles, pero en esencia sigue el mismo esquema

-Tier normalmente se lo suele asociar como una capa física y layer como lógica.

Del libro:

-Mientras más capas se añaden, más se distinguen los elementos que generan datos y los que los presentan.

C/S MULTINIVEL (MULTI-TIER)



-a diferencia de la anterior propuesta se dividió la capa del servidor

COMPONENTES DISTRIBUIDOS

COMPONENTES DISTRIBUIDOS

- El sistema está organizado como un conjunto de **componentes (objetos) que interactúan**.
- El sistema se diseña como un conjunto de servicios, **sin intentar asignar estos servicios a capas específicas** del sistema.
- Cada servicio, o grupo de servicios relacionados, se puede **implementar mediante un componente independiente**.
- Otros componentes recurren a estos servicios a través de **un middleware**, que se encarga de compatibilizar los diferentes componentes (p.ej. tipos)

- esta arquitectura sirvió como base para otras arquitecturas que se usan hoy en día, en cambio las anteriores si siguen usando
- (para este caso toma como componente o objeto, son lo mismo)
- 4) los componentes se comunican entre sí a través de una capa llamada middleware. La idea es que estos componentes se pueden hacer con **diferentes lenguajes** pero debe utilizarse una capa de compatibilidad para poder comunicarse entre sí.

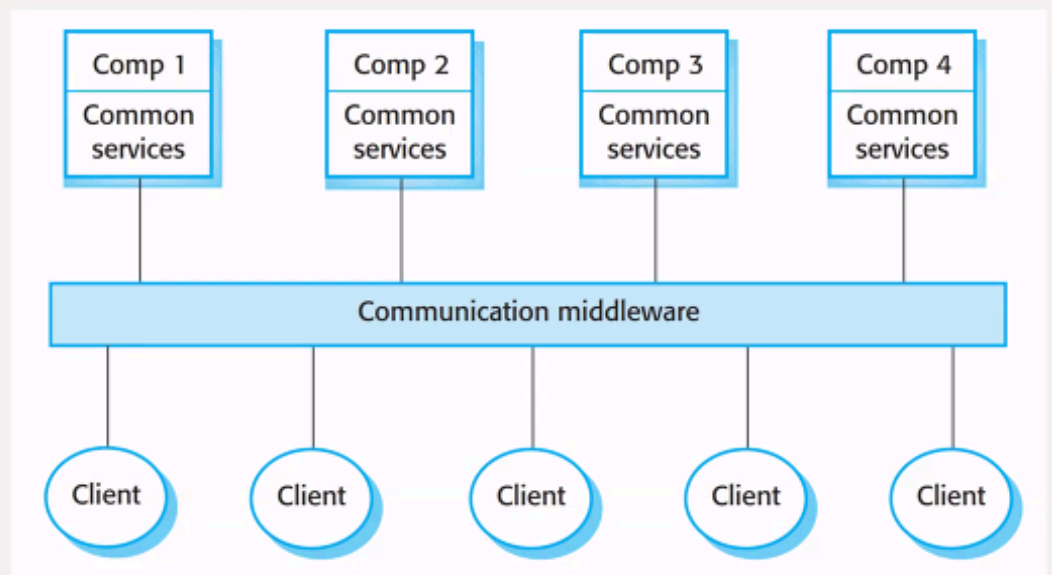
Del libro:

Consiste en separar lógicamente cada capa para que los elementos se dediquen exclusivamente a la capa a la que fueron asignados.

En la teoría funciona bien, pero en la practica es muy difícil saber si el servicio se dedica sola y exclusivamente a eso.

EJEMPLO

COMPONENTES DISTRIBUIDOS



- cada componente se puede desarrollar de manera separada
- los clientes tienen que consumir esos servicios

COMPONENTES DISTRIBUIDOS

• Pros

- Permite **retrasar las decisiones sobre dónde y cómo** deben prestarse los servicios
 - Los nodos pueden ubicarse en cualquier parte de la red
- **Arquitectura muy abierta**, permite agregar nuevos servicios según sea necesario
- **Flexible y escalable**, se pueden replicar componentes según demanda
- **Reconfigurable dinámicamente** con componentes que migran a través de la red según sea necesario

• Contras

- Son **más complejos de diseñar** (menos intuitivo) que los sistemas cliente-servidor.
- **No existen estándares universales** para los modelos de componentes distribuidos o middleware. **Desuso**
 - CORBA (OMG), EJB (Sun/Oracle), DCOM luego WCF .NET (Microsoft)

-con los componentes tenemos que tener bien definidas las interfaces
-esto está basado en un estándar presentado por la OMG, pero al final no tuvo mucho éxito porque cada empresa quiso hacer lo mismo con distintas tecnologías
-en su momento .net intentó realizar algo parecido a CORBA. Dentro de windows se permitía utilizar esta tecnología
-(el autor se refiere sobre todo en exclusivo a CORBA)

-esta arquitectura es la predecesora de lo que vendría ser microservicios

Del libro:

Desventajas: Son mucho más complejos de diseñar, y en la práctica es muy complicado de aplicar algo tan separado en capas.

Por eso es que este tipo de arquitectura es reemplazada por Microservicios

Un ejemplo práctico

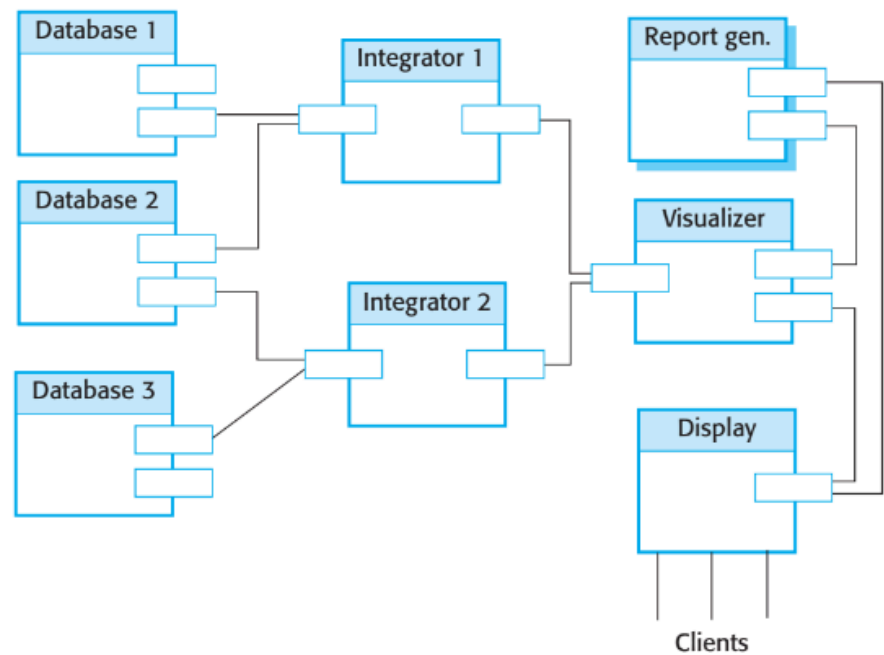


Figure 17.13 A distributed component architecture for a data-mining system

Diferencias

Característica	Arquitectura Cliente-Servidor	Arquitectura de Componentes Distribuidos (DCA)
Estructura	Jerárquica y en capas/niveles. Los roles están claramente definidos: Clientes (consumidores) y Servidores (proveedores).	Red de pares (peers). No hay capas estrictas. Un componente puede actuar como cliente (pidiendo servicios) y como servidor (ofreciéndolos).
Comunicación	Directa. El cliente se comunica directamente con el servidor (o su balanceador de carga). El cliente "sabe" con quién está hablando.	Indirecta (mediada por middleware). Un componente le pide un servicio al "bus" (middleware), y este se encarga de encontrar al componente que lo ofrece.
Acoplamiento	Más acoplada. La comunicación es "vertical" (entre capas). Es más difícil añadir un nuevo servidor que interactúe de forma cruzada.	Muy desacoplada. Los componentes no saben dónde están los otros. Esto permite añadir, quitar o mover componentes dinámicamente.
Complejidad	Más simple e "intuitiva", como dice el texto.	Más compleja de diseñar e implementar, debido a la necesidad de un middleware sofisticado.

PEER TO PEER (P2P) (De igual a igual)

PEER TO PEER (P2P)



- Son sistemas **descentralizados** en los que los cálculos pueden ser realizados por cualquier nodo de la red.
- **No se hacen distinciones entre clientes y servidores.**
- Diseñado para aprovechar la potencia computacional y el almacenamiento disponible en una red de computadoras. Cada nodo debe ejecutar una copia de una aplicación.
- Sistemas personales más que comerciales.
- El hecho de que no haya servidores centrales significa que estos **sistemas son más difíciles de monitorear**; por lo tanto, es posible un mayor nivel de privacidad en las comunicaciones.
- BitTorrent, Bitcoin, SETI@home

-nos permite tener conectados varios nodos y lograr un sistema descentralizado

-aca va un paso más allá, cada nodo se puede ver como un cliente y un servidor al mismo tiempo

-esto se usaba mucho para piratería, no era realmente para eso pero se utilizaba

6) ejemplos que usan esta arquitectura. SETI

-el punto débil era que al ser descentralizado era más difícil de monitorizar

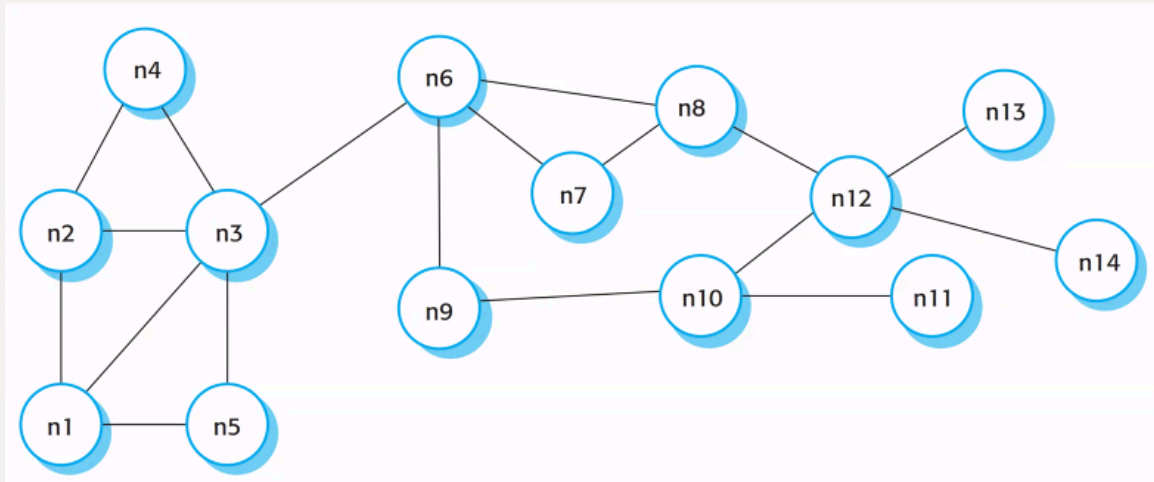
Del libro:

SETI

Fue un proyecto de computación distribuida que permitía a voluntarios usar la potencia de sus computadoras personales para analizar datos de radiotelescopios en busca de inteligencia extraterrestre

El hecho de que no haya un servidor central lo hace especialmente difícil de monitorear y mucho más seguro.

PEER TO PEER (P2P)



-no existen roles predefinidos

PEER TO PEER (P2P)

- Los nodos podrían conectarse e intercambiar datos directamente con cualquier otro nodo de la red. En la práctica, esto es imposible a menos que la red tenga solo unos pocos miembros. Existen localidades y puentes entre localidades
- Los nodos de la red no son simplemente elementos funcionales, sino que también son enrutadores.
- Usable cuando el sistema:
 - Es computacionalmente intensivo (y es posible separar el procesamiento)
 - Implica principalmente el intercambio de información entre computadoras individuales
- La **seguridad** estos sistemas **no se han utilizado ampliamente**. Implican abrir la computadora a interacciones directas con otros peers.

-la idea central es que los nodos se van a conectar entre sí

-se puede llegar a utilizar en productos de cámaras de seguridad en la nube

-esto surgió a fines de los 90, principios de los 2000

-el padre/madre como quieran llamarlo sería la arquitectura orientada de servicios, cuesta mucho diferenciarlo con los componentes distribuidos

Del libro:

Enrutamiento: Los propios nodos actúan como enrutadores. Si un nodo quiere encontrar algo, le pregunta a sus vecinos. Si ellos no lo tienen, le preguntan a sus vecinos, y así sucesivamente hasta que se encuentra el recurso.

Esta arquitectura descentralizada tiene la ventaja de ser altamente redundante y, por lo tanto, tolerante a fallos y a la desconexión de nodos de la red.

Sin embargo, las desventajas son que muchos nodos diferentes pueden procesar la misma búsqueda, y también existe una sobrecarga significativa en las comunicaciones replicadas entre pares.

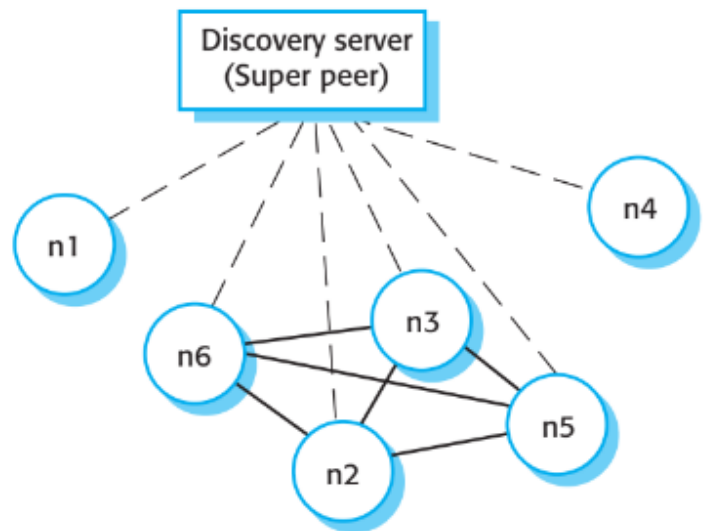


Figure 17.15 A
semicentralized p2p
architecture

Un modelo arquitectónico p2p alternativo, que se aparta de una arquitectura p2p pura, es una arquitectura semicentralizada donde, dentro de la red, uno o más nodos actúan como servidores para facilitar las comunicaciones entre nodos. Esto reduce la cantidad de tráfico entre nodos.

ORIENTADA A SERVICIOS (SOA)

ORIENTADA A SERVICIOS (SOA)

- Los sistemas orientados a servicios son sistemas distribuidos en los que sus **componentes son servicios independientes**, que se ejecutan en equipos distribuidos geográficamente.
 - Los servicios son independientes de la plataforma y del lenguaje de implementación.
- **Servicio:** un acto o ejecución ofrecido por una parte a otra.
 - Aunque el proceso puede estar ligado a un producto físico, la ejecución es esencialmente intangible y normalmente no da lugar a la propiedad de ninguno de los factores de producción.
- **Servicio web:** un componente de software reutilizable y débilmente acoplado que encapsula una funcionalidad discreta, que se puede distribuir y acceder mediante programación.
- Un servicio web es un servicio al que se accede mediante protocolos estándar basados en Internet y XML

BENEFICIOS

ORIENTADA A SERVICIOS (SOA)

- A diferencia de un componente (CBSE), los servicios operan igual, independientemente de su entorno de ejecución. No dependen de componentes externos.
- **WS no tienen una interfaz “requiere”** que implica que otros componentes deban estar presentes. **Simplemente implementa “provee”**.
- Son accedidos por otros programas en vez de usuarios.
- Beneficios:
 - Los servicios pueden ser ofrecidos por **cualquier proveedor** de servicios (int./ext.).
 - **Pueden ser públicos** para que cualquier usuario autorizado pueda utilizar el servicio.
 - Las aplicaciones pueden **cambiar dinámicamente de proveedor** durante la ejecución.
 - Se podría **pagar por los servicios** por uso y no por su compra.
 - Las aplicaciones se pueden hacer más pequeñas (funcionalidad en los servicios).

-en esencia son servicios que pueden ser brindados por cualquier proveedor

- esto fue pensado para que los proveedores fueran públicos y estén disponibles en cualquier parte de la web
- nos abstraemos del concepto de componentes, vamos a estar hablando de servicios independientes
- va a pasar que una aplicación dependa de varios servicios
- como las aplicaciones pueden cambiar de proveedor mientras se está ejecutando. Es el más alto nivel que vamos a tener de diferir enlaces
- como gran parte de sus servicios se encuentran fuera del mismo va a resultar más liviano

ORIENTADA A SERVICIOS (SOA)

- forma de arquitectura de sist. distribuidos caracterizada por (W3C):
 - **Vista lógica:** el servicio es una abstracción definida en términos de lo que hace (operación a nivel empresarial).
 - **Orientación de los mensajes:** se define en términos de los mensajes intercambiados entre los agentes proveedores y los solicitantes, y no de las propiedades de los propios agentes.
 - **Orientación a la descripción:** Un servicio se describe mediante metadatos procesables por máquina.
 - **Granularidad:** tienden a utilizar un pequeño número de operaciones con mensajes relativamente grandes y complejos.
 - **Orientación a la red:** tienden a estar orientadas hacia el uso a través de una red.
 - **Neutral de la plataforma:** los mensajes se envían en un formato estandarizado y neutral de la plataforma que se entregan a través de las interfaces. (XML)

-según W3C, SOA es un tipo de arquitectura de sistemas distribuidos

1) se centra en lo que hace no en cómo lo hace, así que hace que se pueda entender por la gente del negocio también

2) la forma de comunicación es esa en esencia

3) debe ser como autocontenida (autodescribirse), debe tener toda la información necesaria para que otras partes se puedan conectar a ese servicio en particular

ORIENTADA A SERVICIOS (SOA)

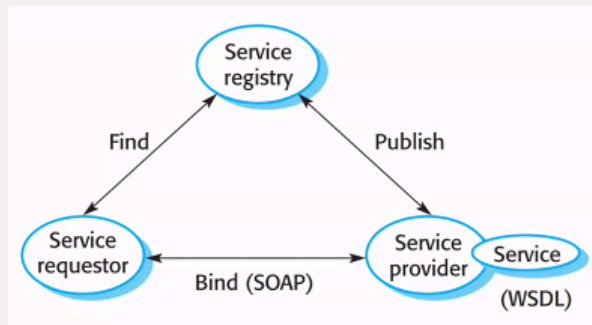
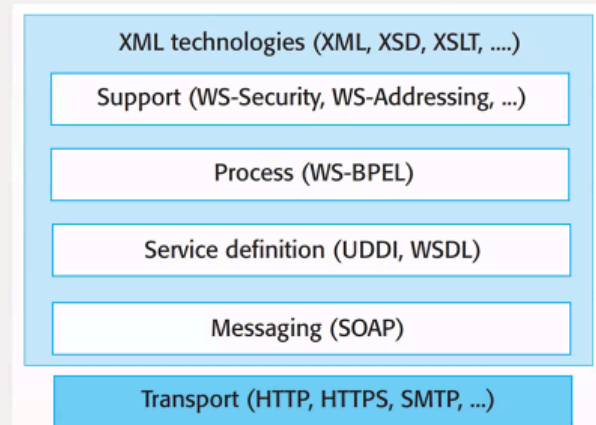


Figure 18.2 Service-oriented architecture

Figure 18.3 Web service standards



- tenemos que tener alguien que haga una petición a ese servicio, pero para realizar la petición primero hay que buscarlo
- se usó por muy poco tiempo esto, el que quería conectarse al servicio lo hacía directamente
- se refiere prácticamente a servicios web
- a la derecha se puede ver las diferentes capas para utilizar este tipo de arquitecturas

2. La Pila de Estándares (Figura 18.3)

El pilar fundamental de SOA es el uso de **estándares abiertos**, lo que garantiza que servicios creados por diferentes empresas, en diferentes lenguajes, puedan interoperar. El texto describe la pila de estándares "clásica" de los servicios web, basada en XML:

- **Transporte (Transport):** La capa base que mueve los mensajes por la red (ej. HTTP, HTTPS).
- **Mensajería (Messaging - SOAP):** Es el "sobre" que envuelve el mensaje. **SOAP** define el formato estándar para empaquetar una solicitud o respuesta.
- **Definición de Servicio (Service definition - WSDL):** Es el "contrato" del servicio. **WSDL** es el lenguaje que describe *exactamente* qué hace un servicio, qué funciones tiene y qué parámetros acepta.
- **Proceso (Process - WS-BPEL):** Es un lenguaje de "orquestración". Permite definir flujos de trabajo complejos que combinan múltiples servicios para lograr un objetivo (ej. una "reserva de viaje" que llama al servicio de vuelos, luego al de hoteles y finalmente al de pagos).
- **Soporte (Support - WS-Security, etc.):** Estándares adicionales para necesidades empresariales como seguridad, transacciones o mensajería confiable.
- **Tecnologías XML:** Todo lo anterior se define usando la familia de tecnologías XML.

ORIENTADA A SERVICIOS (SOA)

- Simple Object Access Protocol (**SOAP**): Se trata de un estándar de **intercambio de mensajes** que admite la comunicación entre servicios. Define los componentes esenciales y opcionales de los mensajes que se pasan entre servicios.
- Web Service Description Language (**WSDL**): es un estándar para la definición de la **interfaz de servicio**. Establece cómo se deben definir las operaciones de servicio (nombres de operación, parámetros y sus tipos) y los enlaces de servicio.
- (Web Services) Business Process Execution Language (**WS-BPEL**): Se trata de un estándar para un lenguaje de **flujo de trabajo** que se utiliza para definir programas de proceso **que implican varios servicios diferentes**.
- Universal Description, Discovery, and Integration (**UDDI**): El estándar de **descubrimiento** define los componentes de una especificación de servicio destinada a ayudar a los usuarios potenciales a descubrir la existencia de un servicio. Desuso.

1) SOAP se encargaba del intercambio de mensajes

2) en los servicios es muy importante la interfaz para saber qué datos necesito

4) UDDI me permitía hacer el descubrimiento de los servicios, era un catálogo de los distintos servicios que existían. Cayó en desuso rápido

-todavía existen implementaciones de servicios web, pero también hoy en día no está muy en boca que digamos

-hay como una especialización de esto de la SOA que son los **Microservicios**

Del libro:

SOA depende fundamentalmente de un estándar, por lo que no sufre incompatibilidades con las innovaciones tecnológicas.

MICROSERVICIOS



- Patrón de arquitectura que se utiliza para implementar arquitecturas lógicas cliente-servidor, donde el servidor se implementa como un conjunto de microservicios que interactúan.
- Los microservicios son servicios a pequeña escala que se pueden combinar para crear aplicaciones.
- Los microservicios son:
 - **Autónomos:** no tienen dependencias externas. Gestionan sus **propios datos** e implementan su propia interfaz de usuario.
 - **Ligero:** comunícate mediante protocolos ligeros, de modo que los gastos generales de comunicación del servicio sean bajos.
 - **Independiente de la implementación:** pueden implementarse utilizando diferentes lenguajes de programación y pueden utilizar diferentes tecnologías (por ejemplo, diferentes tipos de bases de datos) en su implementación.
 - **Desplegable de forma independiente:** Cada uno se ejecuta en su propio proceso y se puede desplegar de forma independiente, utilizando sistemas automatizados.
 - **Orientado al negocio:** debe implementar las capacidades y necesidades del negocio, en lugar de simplemente proporcionar un servicio técnico.

-son difíciles de diferenciarlo de una arquitectura SOA

-la escala debería ser más pequeña incluso que la arquitectura SOA, esto es porque es muy importante que sean autónomos

-se puede relacionar con el primer principio de SOLID de Responsabilidad Única

-es una arquitectura muy escalable, pero a su vez es más compleja de manejar porque tenemos más módulos que manejar

(fijense que estuvimos dando un conjunto de patrones, esto es un catálogo de los patrones más comunes. Del libro de Ingeniería de SW de Somerville)

(los otros autores(Bass) mencionan patrones en el contexto de Atributos de calidad)

(revisar los patrones que fuimos nombrando si aparecen en el libro de Bass y a cual se asocia, ver si aporta algo respecto a ese patrón)

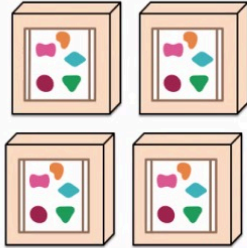
(Microservicios si aparece en el libro de Bass)

MICROSERVICIOS

A monolithic application puts all its functionality into a single process...



... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.

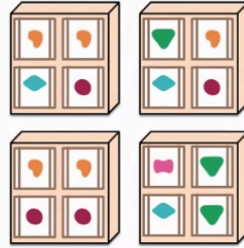


Figure 1: Monoliths and Microservices

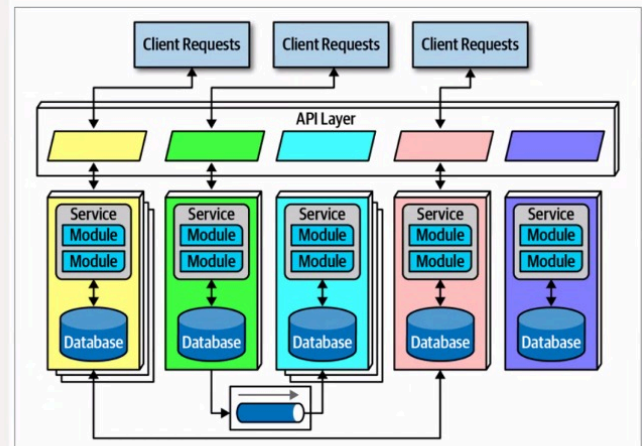
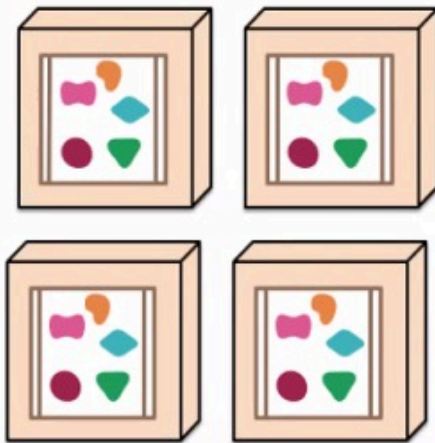


Figure 17-1. The topology of the microservices architecture style

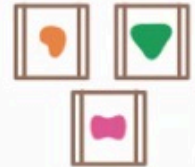
A monolithic application puts all its functionality into a single process...



... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.

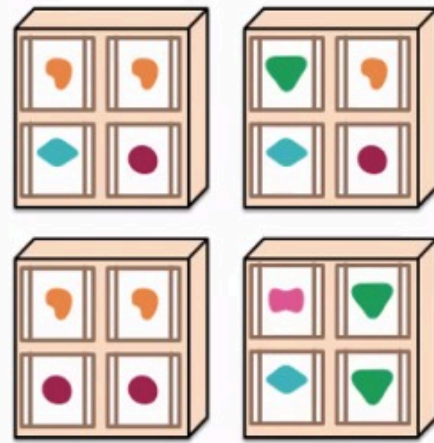


Figure 1: Monoliths and Microservices

- diagrama izquierda
- muestra lo que hoy en día se considera una arquitectura monolítica
- cada elemento va a poder trabajar de manera autónoma
- al final al cabo lo que se hace es hacer el despliegue de estos nodos

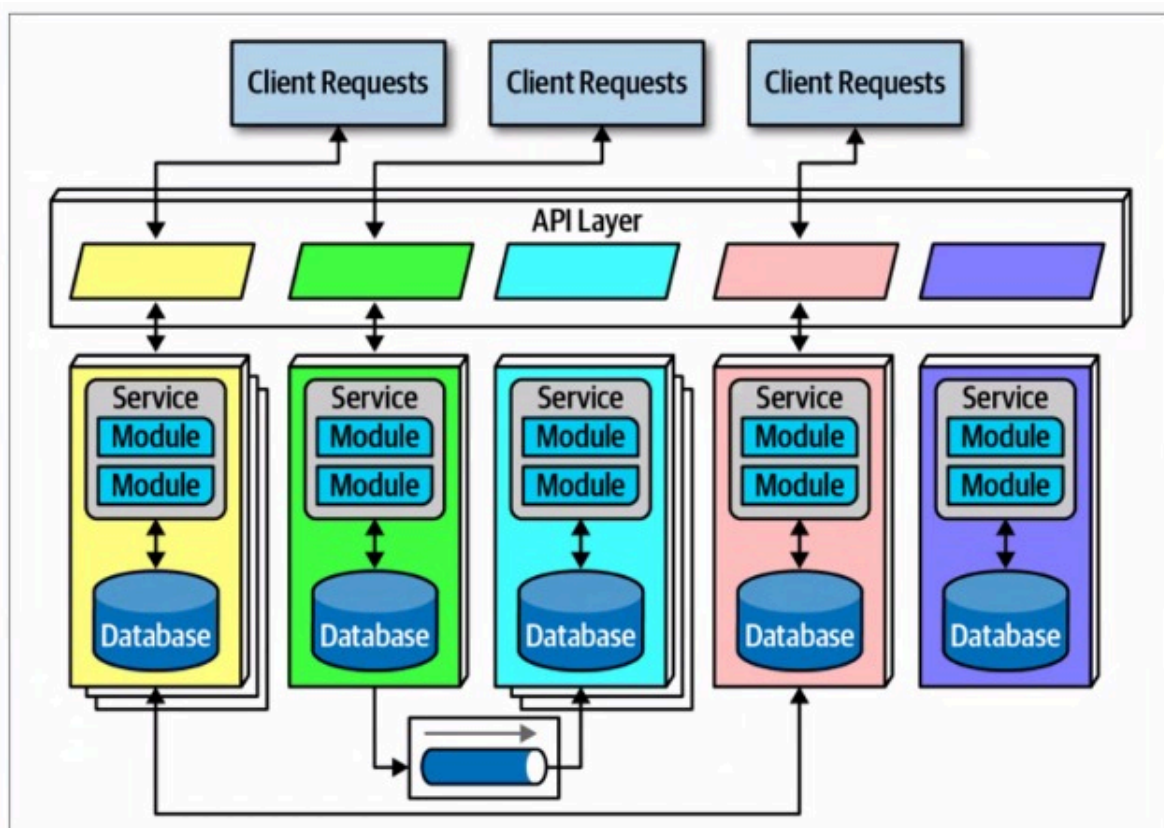


Figure 17-1. The topology of the microservices architecture style

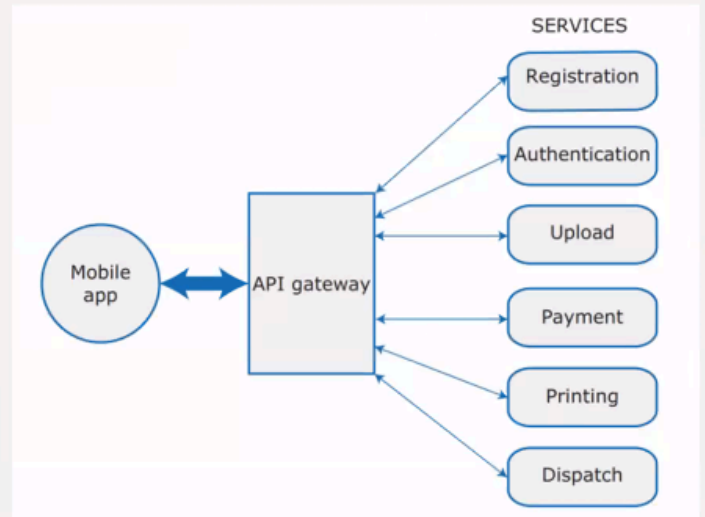
-- diagrama derecha

-al estar todo tan dividido necesitamos una capa que junte todo esos microservicios que vamos a tener ahí, generalmente llamada capa API

MICROSERVICIOS

- **Servicio de impresión de fotos**

- Los usuarios pueden subir fotos a su servidor desde su teléfono o especificar fotos de su cuenta de Instagram que les gustaría imprimir.
- Las impresiones pueden hacerse en distintos tamaños y soportes.
- Se envían a domicilio.
- Pagos por Android o Apple Pay o registrando una tarjeta de crédito con el proveedor de servicios de impresión.



(la idea es que investigamos más acerca de este tema)

(recomendó que vayamos a esta página del artículo de Fowler)

(dijo que esto **no** se encuentra en el capítulo 17 de Somerville)

(Arquitectura es un tema central para la materia, lo vemos de manera explícita. Va a preguntar mucho en el parcial)

Software as a service (SaaS)

Gaona no hizo hincapié en SAAS en clase, pero está bien saber que NO es lo mismo que SOA (Service Oriented Software, cap 18)

SaaS no es un patrón de arquitectura en sí mismo, sino un modelo de entrega y negocio.

La idea de SAAS es hostear un software de forma remota y que los usuarios accedan desde internet.

Por lo general se despliega en la nube, no en una PC local.

El software es controlado por el dueño del mismo más que por las organizaciones que lo usan.

A veces hay que pagar suscripción o es gratis (aceptando términos), depende del distribuidor.

Los beneficios más destacados son:

- No se pagan más licencias por tener software en distintos dispositivos porque no se ejecuta directamente en dispositivos distintos.
- Es más barato actualizar y arreglar bugs porque actualizas solamente el servicio en la nube.
- Si un cliente tiene distintas computadoras, no necesita pagar licencia en todas para poder utilizar el software, basta con tener cuenta habilitada.

Desventajas:

La principal desventaja es el uso de red, debido a que todo es virtual se produce una gran carga en la red de transferencia de datos.

Otra desventaja es que tenés (como cliente) que lidiar con las actualizaciones y evolución sin control del software.

SaaS se implementa utilizando componentes mas que utilizando servicios

Algunos SaaS como Google Docs para usuarios web ofrecen experiencias genéricas para todos los usuarios.

Hay empresas que desean tener versiones específicas que se adapten a sus necesidades.

Como es muy caro hacer un software para cada empresa, lo ideal es desarrollar una buena configuración para que cada uno lo adapte a sus necesidades.

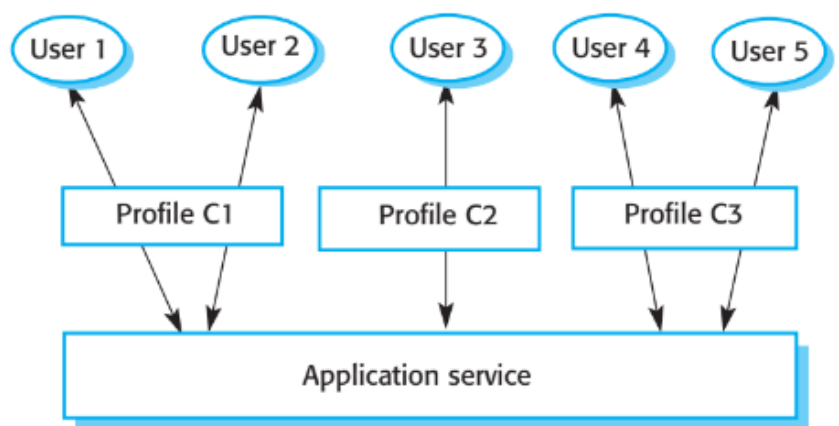


Figure 17.16
Configuration of a
software system
offered as a service